Qryx - Shopify AI Sales Assistant

Enterprise-Grade AI Chatbot for Shopify Stores

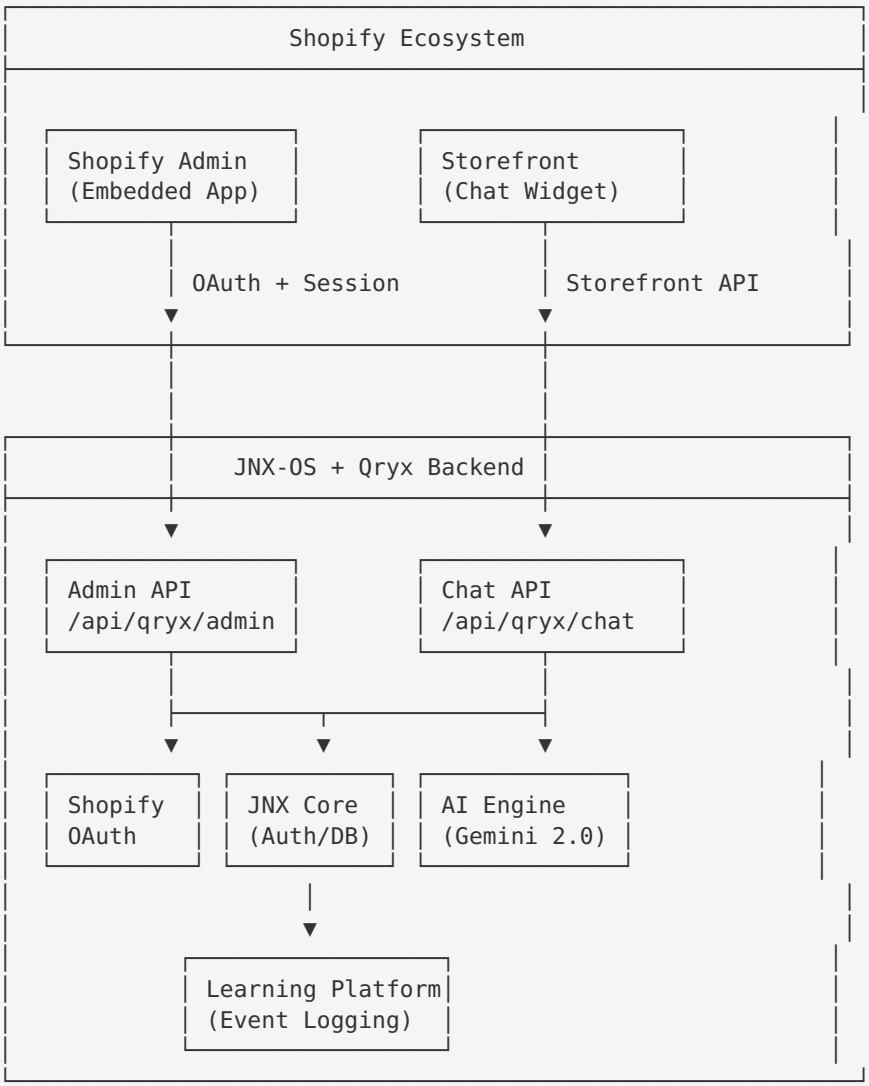
Last Updated: 2024-12-28
Status: Planning Phase
Type: Shopify Embedded App + Storefront Widget

🎯 Executive Summary

Qryx ist ein AI-powered Sales Assistant für Shopify-Shops, der:
- **In Shopify Admin** eingebettet ist (Embedded App)
- **Im Storefront** als Chat-Widget für Kunden verfügbar ist
- **Von JNX-OS** powered wird (Auth, DB, Learning Platform)
- **One-Click Installation** aus dem Shopify App Store ermöglicht
- **Multi-Tenant** (jeder Shop = eigener Tenant)
- **Monetarisiert** via Shopify Billing API

🏗️ Architecture Overview

High-Level Architecture



Key Components

1. Shopify Admin App (Embedded)
- Dashboard mit Chat-Analytics
- Konfiguration (Prompts, Produkt-Katalog)
- Chat History & Insights
- Billing Management

2. Storefront Widget
- Floating Chat Button
- Chat-Interface für Kunden
- Product Recommendations
- Order Tracking

3. Backend (JNX-OS)
- Shopify OAuth Handler
- Multi-Tenant Shop Management

- AI Chat Engine (Gemini 2.0 Flash)
- Learning Platform Integration
- Billing API Integration

🗄️ Database Schema

New Tables

```sql
-- ============================================
-- Qryx Shopify Integration Schema
-- ============================================

-- Shopify Shops (each shop is a tenant)
CREATE TABLE IF NOT EXISTS shopify_shops (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  shop_domain VARCHAR(255) NOT NULL UNIQUE,  -- myshop.myshopify.com
  shop_name VARCHAR(255),
  shop_email VARCHAR(255),

  -- Shopify OAuth
  access_token TEXT NOT NULL,  -- Encrypted
  scope TEXT NOT NULL,

  -- Shop Info
  shop_owner VARCHAR(255),
  plan_name VARCHAR(100),  -- Basic, Shopify, Advanced, Plus
  currency VARCHAR(10) DEFAULT 'USD',
  timezone VARCHAR(100),

  -- Installation Status
  installed_at TIMESTAMPTZ DEFAULT NOW(),
  uninstalled_at TIMESTAMPTZ,
  status VARCHAR(50) DEFAULT 'active',  -- active, suspended, uninstalled

  -- Billing
  subscription_id VARCHAR(255),  -- Shopify Billing API
  plan_tier VARCHAR(50) DEFAULT 'trial',  -- trial, basic, pro, enterprise
  trial_ends_at TIMESTAMPTZ,

  -- Metadata
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_shops_domain ON shopify_shops(shop_domain);
CREATE INDEX idx_shops_status ON shopify_shops(status);
CREATE INDEX idx_shops_plan_tier ON shopify_shops(plan_tier);

-- ============================================

-- Qryx Chat Sessions
CREATE TABLE IF NOT EXISTS qryx_chat_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  shop_id UUID NOT NULL REFERENCES shopify_shops(id) ON DELETE CASCADE,

  -- Customer Info
  customer_id VARCHAR(255),  -- Shopify Customer ID (if logged in)
  customer_email VARCHAR(255),
  customer_name VARCHAR(255),
  session_token TEXT NOT NULL,  -- Anonymous session ID

  -- Session Metadata
  started_at TIMESTAMPTZ DEFAULT NOW(),
  ended_at TIMESTAMPTZ,
  message_count INTEGER DEFAULT 0,

  -- Context
  page_url TEXT,
  referrer TEXT,
```

```
  device_type VARCHAR(50),  -- desktop, mobile, tablet

  -- Outcomes
  has_order BOOLEAN DEFAULT FALSE,
  order_id VARCHAR(255),
  order_value DECIMAL(10,2),

  -- Analytics
  satisfaction_rating INTEGER,  -- 1-5
  tags TEXT[],

  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_sessions_shop ON qryx_chat_sessions(shop_id);
CREATE INDEX idx_sessions_customer ON qryx_chat_sessions(customer_id);
CREATE INDEX idx_sessions_started ON qryx_chat_sessions(started_at DESC);

-- ============================================

-- Qryx Chat Messages
CREATE TABLE IF NOT EXISTS qryx_chat_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID NOT NULL REFERENCES qryx_chat_sessions(id) ON DELETE CASCADE,
  shop_id UUID NOT NULL REFERENCES shopify_shops(id) ON DELETE CASCADE,

  -- Message Content
  role VARCHAR(20) NOT NULL,  -- user, assistant, system
  content TEXT NOT NULL,

  -- AI Metadata
  model VARCHAR(100),  -- gemini-2.0-flash-exp
  tokens_used INTEGER,
  response_time_ms INTEGER,

  -- Context
  intent VARCHAR(100),  -- product_inquiry, order_tracking, support, etc.
  confidence_score DECIMAL(3,2),

  -- Actions
  products_shown JSONB DEFAULT '[]',  -- [product_id, product_id]
  action_taken VARCHAR(100),  -- showed_product, created_cart, etc.

  -- Feedback
  feedback VARCHAR(20),  -- positive, negative, neutral

  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_messages_session ON qryx_chat_messages(session_id);
CREATE INDEX idx_messages_shop ON qryx_chat_messages(shop_id);
CREATE INDEX idx_messages_created ON qryx_chat_messages(created_at DESC);
CREATE INDEX idx_messages_role ON qryx_chat_messages(role);

-- ============================================

-- Qryx Configuration (per shop)
CREATE TABLE IF NOT EXISTS qryx_config (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  shop_id UUID NOT NULL UNIQUE REFERENCES shopify_shops(id) ON DELETE CASCADE,

  -- Chatbot Behavior
  bot_name VARCHAR(100) DEFAULT 'Qryx',
```

```
  bot_greeting TEXT DEFAULT 'Hi! How can I help you today?',
  bot_personality TEXT DEFAULT 'friendly',  -- friendly, professional, casual
  response_style VARCHAR(50) DEFAULT 'balanced',  -- concise, balanced, detailed

  -- Features
  product_recommendations_enabled BOOLEAN DEFAULT TRUE,
  order_tracking_enabled BOOLEAN DEFAULT TRUE,
  live_chat_fallback_enabled BOOLEAN DEFAULT FALSE,

  -- Prompts
  system_prompt TEXT,
  product_context_prompt TEXT,

  -- UI Customization
  theme_color VARCHAR(7) DEFAULT '#0ea5e9',  -- Hex color
  position VARCHAR(20) DEFAULT 'bottom-right',  -- bottom-right, bottom-left
  widget_enabled BOOLEAN DEFAULT TRUE,

  -- Business Hours
  business_hours JSONB DEFAULT '{"enabled": false}',

  -- Limits (per plan)
  monthly_message_limit INTEGER DEFAULT 1000,

  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_config_shop ON qryx_config(shop_id);

-- ============================================

-- Qryx Analytics (aggregated daily)
CREATE TABLE IF NOT EXISTS qryx_analytics (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  shop_id UUID NOT NULL REFERENCES shopify_shops(id) ON DELETE CASCADE,
  date DATE NOT NULL,

  -- Volume
  sessions_started INTEGER DEFAULT 0,
  messages_sent INTEGER DEFAULT 0,
  unique_customers INTEGER DEFAULT 0,

  -- Engagement
  avg_messages_per_session DECIMAL(5,2),
  avg_session_duration_seconds INTEGER,

  -- Conversion
  sessions_with_order INTEGER DEFAULT 0,
  total_order_value DECIMAL(10,2) DEFAULT 0,
  conversion_rate DECIMAL(5,4),

  -- Satisfaction
  positive_feedback_count INTEGER DEFAULT 0,
  negative_feedback_count INTEGER DEFAULT 0,
  avg_satisfaction_rating DECIMAL(3,2),

  -- Performance
  avg_response_time_ms INTEGER,

  created_at TIMESTAMPTZ DEFAULT NOW(),

  UNIQUE(shop_id, date)
```

```
);

CREATE INDEX idx_analytics_shop_date ON qryx_analytics(shop_id, date DESC);
```

Integration with Existing Schema

Connection zu JNX-OS:
- `shopify_shops.id` → kann mit `orgs` verknüpft werden (optional)
- Shop Owner kann als `users` entry angelegt werden
- Events werden via JNX Learning Platform geloggt

🔐 Authentication Flow

Shopify OAuth 2.0

```
1. Installation Start
   User clicks "Add App" in Shopify App Store
   ↓
2. Redirect to OAuth
   GET /api/qryx/auth/shopify?shop={shop-domain}
   ↓
3. User Grants Permissions
   Shopify redirects to callback URL
   ↓
4. Exchange Code for Token
   POST /api/qryx/auth/shopify/callback
   - Validates HMAC
   - Exchanges code for access_token
   - Stores in database (encrypted)
   ↓
5. Create Shop Record
   - Insert into shopify_shops
   - Create default qryx_config
   - Start trial period
   ↓
6. Redirect to App Dashboard
   Embedded in Shopify Admin
```

Required Scopes

```
const SHOPIFY_SCOPES = [
  'read_products',           // Product catalog access
  'read_orders',             // Order tracking
  'read_customers',          // Customer data (with consent)
  'write_script_tags',       // Inject storefront widget
  'read_content',            // Shop info
];
```

Session Management

Two types of sessions:

1. **Admin Session** (Shop Owner in Shopify Admin)
   - Shopify Session Token (JWT)

- Validated via `@shopify/shopify-api`
- Short-lived (few minutes)

2. **Storefront Session** (Customers)
   - Anonymous Session ID (UUID)
   - Stored in localStorage
   - Linked to Shopify Customer ID (if logged in)

📦 Installation Flow

One-Click Installation

Step 1: App Listing (Shopify App Store)

```
{
  "name": "Qryx - AI Sales Assistant",
  "tagline": "AI-powered chatbot that sells",
  "pricing": "Free trial, paid plans from $29/month",
  "oauth_redirect_uri": "https://jnx-os.app/api/qryx/auth/shopify/callback",
  "scopes": ["read_products", "read_orders", ...]
}
```

Step 2: Installation Webhook

```
// app/api/qryx/auth/shopify/install/route.ts
export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const shop = searchParams.get('shop');

  // 1. Validate shop domain
  if (!isValidShopDomain(shop)) {
    return new Response('Invalid shop', { status: 400 });
  }

  // 2. Generate OAuth URL
  const authUrl = generateShopifyAuthUrl(shop, {
    scopes: SHOPIFY_SCOPES,
    redirectUri: `${process.env.APP_URL}/api/qryx/auth/shopify/callback`,
    state: generateState() // CSRF protection
  });

  // 3. Redirect to Shopify OAuth
  return Response.redirect(authUrl);
}
```

Step 3: Callback Handler

```
// app/api/qryx/auth/shopify/callback/route.ts
export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const shop = searchParams.get('shop');
  const code = searchParams.get('code');
  const hmac = searchParams.get('hmac');
  const state = searchParams.get('state');

  // 1. Validate HMAC
  if (!validateHMAC(searchParams, hmac)) {
    return new Response('Invalid HMAC', { status: 403 });
  }

  // 2. Validate state (CSRF)
  if (!validateState(state)) {
    return new Response('Invalid state', { status: 403 });
  }

  // 3. Exchange code for access token
  const tokenResponse = await fetch(
    `https://${shop}/admin/oauth/access_token`,
    {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        client_id: process.env.SHOPIFY_API_KEY,
        client_secret: process.env.SHOPIFY_API_SECRET,
        code
      })
    }
  );

  const { access_token, scope } = await tokenResponse.json();

  // 4. Fetch shop details
  const shopDetails = await fetchShopDetails(shop, access_token);

  // 5. Create/Update shop record
  const shopRecord = await upsertShop({
    shop_domain: shop,
    shop_name: shopDetails.name,
    shop_email: shopDetails.email,
    access_token: encryptToken(access_token),
    scope,
    installed_at: new Date(),
    status: 'active',
    plan_tier: 'trial',
    trial_ends_at: addDays(new Date(), 14)
  });

  // 6. Create default config
  await createDefaultConfig(shopRecord.id);

  // 7. Install storefront widget (Script Tag API)
  await installStorefrontWidget(shop, access_token);

  // 8. Log event to Learning Platform
  await logProductEvent('qryx', 'shop_installed', {
    shop_id: shopRecord.id,
    shop_domain: shop,
    plan: shopDetails.plan_name
  });
```

```
  // 9. Redirect to embedded app
  return Response.redirect(
    `https://${shop}/admin/apps/${process.env.SHOPIFY_APP_HANDLE}`
  );
}
```

Step 4: Widget Installation

```
async function installStorefrontWidget(
  shop: string,
  accessToken: string
) {
  // Install script tag
  await fetch(`https://${shop}/admin/api/2024-01/script_tags.json`, {
    method: 'POST',
    headers: {
      'X-Shopify-Access-Token': accessToken,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      script_tag: {
        event: 'onload',
        src: `${process.env.APP_URL}/qryx-widget.js?shop=${shop}`
      }
    })
  });
}
```

💬 Chat Implementation

Admin Interface (Embedded App)

Location: `/app/qryx/admin/page.tsx`

```
'use client';

import { useEffect, useState } from 'react';
import { useSearchParams } from 'next/navigation';

interface ShopData {
  shop: string;
  // ... analytics data
}

export default function QryxAdminDashboard() {
  const searchParams = useSearchParams();
  const shop = searchParams.get('shop');
  const [data, setData] = useState<ShopData | null>(null);

  useEffect(() => {
    // Fetch shop data
    async function loadShopData() {
      const response = await fetch(
        `/api/qryx/admin/dashboard?shop=${shop}`,
        {
          headers: {
            'Authorization': `Bearer ${getSessionToken()}`
          }
        }
      );
      const data = await response.json();
      setData(data);
    }

    if (shop) loadShopData();
  }, [shop]);

  return (
    <div className="p-6">
      <h1>Qryx Dashboard</h1>
      {/* Analytics, Config, Chat History */}
    </div>
  );
}
```

Features:
- 📊 Dashboard mit Analytics
- ⚙️ Configuration (Prompts, Styling)
- 💬 Chat History & Transcripts
- 📈 Conversion Tracking
- 💳 Billing Management

Storefront Widget

File: `/public/qryx-widget.js`

```
(function() {
  // 1. Extract shop domain from script src
  const scriptTag = document.currentScript;
  const shop = new URL(scriptTag.src).searchParams.get('shop');

  // 2. Create chat widget
  const widget = document.createElement('div');
  widget.id = 'qryx-chat-widget';
  widget.innerHTML = `
    <div id="qryx-button" class="qryx-button">
      <svg>...</svg>
    </div>
    <div id="qryx-window" class="qryx-window" style="display:none">
      <div class="qryx-header">
        <span>Chat with us</span>
        <button id="qryx-close">×</button>
      </div>
      <div id="qryx-messages" class="qryx-messages"></div>
      <div class="qryx-input">
        <input type="text" id="qryx-input" placeholder="Type a message..." />
        <button id="qryx-send">Send</button>
      </div>
    </div>
  `;

  // 3. Load styles
  const styles = document.createElement('link');
  styles.rel = 'stylesheet';
  styles.href = `https://jnx-os.app/qryx-widget.css`;
  document.head.appendChild(styles);

  // 4. Append to body
  document.body.appendChild(widget);

  // 5. Initialize chat
  window.QryxChat = new QryxChatClient({
    shop,
    apiUrl: 'https://jnx-os.app/api/qryx/chat',
    sessionId: getOrCreateSessionId()
  });

  // 6. Event listeners
  document.getElementById('qryx-button').onclick = () => {
    document.getElementById('qryx-window').style.display = 'flex';
    window.QryxChat.startSession();
  };

  // ... more event handlers
})();

// Chat Client Class
class QryxChatClient {
  constructor(config) {
    this.shop = config.shop;
    this.apiUrl = config.apiUrl;
    this.sessionId = config.sessionId;
    this.messages = [];
  }

  async sendMessage(text) {
    const response = await fetch(this.apiUrl, {
      method: 'POST',
```

```
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        shop: this.shop,
        session_id: this.sessionId,
        message: text
      })
    });

    const data = await response.json();
    this.addMessage('assistant', data.response);
    return data;
  }

  addMessage(role, content) {
    this.messages.push({ role, content });
    this.renderMessages();
  }

  renderMessages() {
    const container = document.getElementById('qryx-messages');
    container.innerHTML = this.messages.map(msg => `
      <div class="qryx-message qryx-message-${msg.role}">
        ${msg.content}
      </div>
    `).join('');
    container.scrollTop = container.scrollHeight;
  }

  startSession() {
    // Log session start
    fetch(`${this.apiUrl}/session/start`, {
      method: 'POST',
      body: JSON.stringify({
        shop: this.shop,
        session_id: this.sessionId,
        page_url: window.location.href
      })
    });
  }
}

function getOrCreateSessionId() {
  let sessionId = localStorage.getItem('qryx_session_id');
  if (!sessionId) {
    sessionId = crypto.randomUUID();
    localStorage.setItem('qryx_session_id', sessionId);
  }
  return sessionId;
}
```

Chat API Endpoint

File: `/app/api/qryx/chat/route.ts`

```
import { NextRequest } from 'next/server';
import { GoogleGenerativeAI } from '@google/generative-ai';
import { useProductLogger } from '@/lib/jnx-products';

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY!);

export async function POST(request: NextRequest) {
  try {
    const body = await request.json();
    const { shop, session_id, message } = body;

    // 1. Validate shop
    const shopRecord = await getShopByDomain(shop);
    if (!shopRecord || shopRecord.status !== 'active') {
      return Response.json(
        { error: 'Shop not found or inactive' },
        { status: 403 }
      );
    }

    // 2. Check message limits
    if (await isMessageLimitExceeded(shopRecord.id)) {
      return Response.json(
        { error: 'Message limit exceeded. Please upgrade your plan.' },
        { status: 429 }
      );
    }

    // 3. Get session context
    const session = await getOrCreateSession(
      shopRecord.id,
      session_id
    );

    // 4. Get chat history
    const history = await getChatHistory(session.id);

    // 5. Get shop context (products, config)
    const shopContext = await buildShopContext(shopRecord);

    // 6. Build prompt
    const systemPrompt = `
You are ${shopContext.bot_name}, an AI sales assistant for ${shopContext.shop_name}.

Personality: ${shopContext.bot_personality}
Response Style: ${shopContext.response_style}

Product Catalog:
${shopContext.products.map(p => `- ${p.title}: $${p.price}`).join('\n')}

${shopContext.system_prompt || ''}

Always be helpful, accurate, and encourage purchases when appropriate.
`;

    // 7. Call Gemini
    const model = genAI.getGenerativeModel({ model: 'gemini-2.0-flash-exp' });
    const startTime = Date.now();

    const chat = model.startChat({
      history: history.map(msg => ({
        role: msg.role === 'assistant' ? 'model' : 'user',
```

```
        parts: [{ text: msg.content }]
      })),
      generationConfig: {
        maxOutputTokens: 1000,
        temperature: 0.7
      }
    });

    const result = await chat.sendMessage([
      { text: systemPrompt },
      { text: message }
    ]);

    const responseTime = Date.now() - startTime;
    const response = result.response.text();

    // 8. Save messages
    await saveMessage({
      session_id: session.id,
      shop_id: shopRecord.id,
      role: 'user',
      content: message
    });

    await saveMessage({
      session_id: session.id,
      shop_id: shopRecord.id,
      role: 'assistant',
      content: response,
      model: 'gemini-2.0-flash-exp',
      response_time_ms: responseTime
    });

    // 9. Log to Learning Platform
    const logger = useProductLogger('qryx');
    await logger.logEvent('chat_message', {
      shop_id: shopRecord.id,
      session_id: session.id,
      message_length: message.length,
      response_time: responseTime,
      model: 'gemini-2.0-flash-exp'
    });

    // 10. Return response
    return Response.json({
      response,
      session_id: session.id,
      products: extractProductRecommendations(response, shopContext.products)
    });

  } catch (error) {
    console.error('Chat error:', error);
    return Response.json(
      { error: 'Failed to process message' },
      { status: 500 }
    );
  }
}
```

🎨 UI Components

Admin Dashboard Components

1. Analytics Overview

```
// components/qryx/analytics-overview.tsx
export function AnalyticsOverview({ shopId }: { shopId: string }) {
  const [analytics, setAnalytics] = useState<Analytics | null>(null);

  useEffect(() => {
    fetch(`/api/qryx/admin/analytics?shop_id=${shopId}`)
      .then(r => r.json())
      .then(setAnalytics);
  }, [shopId]);

  if (!analytics) return <Skeleton />;

  return (
    <div className="grid grid-cols-4 gap-4">
      <Card>
        <CardHeader>
          <CardTitle>Sessions</CardTitle>
        </CardHeader>
        <CardContent>
          <div className="text-3xl font-bold">{analytics.sessions_today}</div>
          <p className="text-sm text-muted">+{analytics.sessions_change}% from yester-
day</p>
        </CardContent>
      </Card>

      <Card>
        <CardTitle>Messages</CardTitle>
        <CardContent>
          <div className="text-3xl">{analytics.messages_today}</div>
        </CardContent>
      </Card>

      <Card>
        <CardTitle>Conversion Rate</CardTitle>
        <CardContent>
          <div className="text-3xl">{analytics.conversion_rate}%</div>
        </CardContent>
      </Card>

      <Card>
        <CardTitle>Revenue Generated</CardTitle>
        <CardContent>
          <div className="text-3xl">${analytics.revenue_generated}</div>
        </CardContent>
      </Card>
    </div>
  );
}
```

2. Configuration Panel

```tsx
// components/qryx/config-panel.tsx
export function ConfigPanel({ shopId }: { shopId: string }) {
  const [config, setConfig] = useState<QryxConfig | null>(null);

  const handleSave = async () => {
    await fetch('/api/qryx/admin/config', {
      method: 'PUT',
      body: JSON.stringify(config)
    });
  };

  return (
    <div className="space-y-6">
      <Card>
        <CardHeader>
          <CardTitle>Chatbot Behavior</CardTitle>
        </CardHeader>
        <CardContent>
          <div className="space-y-4">
            <InputField
              label="Bot Name"
              value={config?.bot_name}
              onChange={(e) => setConfig({ ...config!, bot_name: e.target.value })}
            />

            <Textarea
              label="Greeting Message"
              value={config?.bot_greeting}
              onChange={(e) => setConfig({ ...config!, bot_greeting: e.target.value })
}
            />

            <Select
              label="Personality"
              value={config?.bot_personality}
              options={[
                { value: 'friendly', label: 'Friendly' },
                { value: 'professional', label: 'Professional' },
                { value: 'casual', label: 'Casual' }
              ]}
            />
          </div>
        </CardContent>
      </Card>

      <ButtonPrimary onClick={handleSave}>Save Changes</ButtonPrimary>
    </div>
  );
}
```

3. Chat History

```tsx
// components/qryx/chat-history.tsx
export function ChatHistory({ shopId }: { shopId: string }) {
  const [sessions, setSessions] = useState<ChatSession[]>([]);

  return (
    <div>
      <Table>
        <TableHeader>
          <TableRow>
            <TableHead>Customer</TableHead>
            <TableHead>Started</TableHead>
            <TableHead>Messages</TableHead>
            <TableHead>Order</TableHead>
            <TableHead>Satisfaction</TableHead>
          </TableRow>
        </TableHeader>
        <TableBody>
          {sessions.map(session => (
            <TableRow key={session.id}>
              <TableCell>{session.customer_name || 'Anonymous'}</TableCell>
              <TableCell>{formatDate(session.started_at)}</TableCell>
              <TableCell>{session.message_count}</TableCell>
              <TableCell>
                {session.has_order ? (
                  <Badge variant="success">${session.order_value}</Badge>
                ) : (
                  <Badge variant="secondary">No order</Badge>
                )}
              </TableCell>
              <TableCell>
                {session.satisfaction_rating ? (
                  <StarRating rating={session.satisfaction_rating} />
                ) : (
                  <span className="text-muted">-</span>
                )}
              </TableCell>
            </TableRow>
          ))}
        </TableBody>
      </Table>
    </div>
  );
}
```

🎯 JNX Learning Platform Integration

Qryx Product Configuration

File: `/lib/jnx-products/qryx/config.ts`

```
import { z } from 'zod';
import { defineProduct } from '@/lib/jnx-core/registry';

export default defineProduct({
  id: 'qryx',
  name: 'Qryx AI Sales Assistant',
  version: '1.0.0',

  events: {
    // Installation Events
    'shop_installed': {
      schema: z.object({
        shop_id: z.string().uuid(),
        shop_domain: z.string(),
        plan: z.string()
      }),
      description: 'When a shop installs Qryx'
    },

    'shop_uninstalled': {
      schema: z.object({
        shop_id: z.string().uuid(),
        shop_domain: z.string(),
        reason: z.string().optional()
      }),
      description: 'When a shop uninstalls Qryx'
    },

    // Chat Events
    'chat_session_started': {
      schema: z.object({
        shop_id: z.string().uuid(),
        session_id: z.string().uuid(),
        customer_id: z.string().optional(),
        page_url: z.string()
      }),
      description: 'When a customer starts a chat'
    },

    'chat_message': {
      schema: z.object({
        shop_id: z.string().uuid(),
        session_id: z.string().uuid(),
        message_length: z.number().positive(),
        response_time: z.number().positive(),
        model: z.string(),
        intent: z.string().optional(),
        products_shown: z.array(z.string()).optional()
      }),
      description: 'When a message is sent/received'
    },

    'chat_feedback': {
      schema: z.object({
        shop_id: z.string().uuid(),
        session_id: z.string().uuid(),
        message_id: z.string().uuid(),
        feedback: z.enum(['positive', 'negative', 'neutral'])
      }),
      description: 'When user provides feedback'
    },
```

```
    'chat_session_ended': {
      schema: z.object({
        shop_id: z.string().uuid(),
        session_id: z.string().uuid(),
        duration_seconds: z.number().positive(),
        message_count: z.number().nonnegative(),
        has_order: z.boolean(),
        order_value: z.number().optional()
      }),
      description: 'When a chat session ends'
    },

    // Conversion Events
    'conversion_tracked': {
      schema: z.object({
        shop_id: z.string().uuid(),
        session_id: z.string().uuid(),
        order_id: z.string(),
        order_value: z.number().positive(),
        products: z.array(z.string())
      }),
      description: 'When a chat leads to a purchase'
    },

    // Configuration Events
    'config_updated': {
      schema: z.object({
        shop_id: z.string().uuid(),
        changes: z.record(z.any())
      }),
      description: 'When shop owner updates config'
    }
  },

  // Protected Paths (AI can't modify)
  protected: [
    'api/qryx/auth/*',           // OAuth flow
    'api/qryx/billing/*',        // Billing
    'core/shopify-integration',  // Shopify API
    'core/security',             // Security
    'webhooks/*'                 // Shopify webhooks
  ],

  // Optimizable Paths (AI can suggest improvements)
  optimizable: [
    'prompts/system',            // System prompts
    'prompts/product-context',   // Product descriptions
    'ui/widget-styling',         // Widget appearance
    'ui/greeting-messages',      // First message
    'performance/caching',       // Response caching
    'recommendations/logic'      // Product recommendation algorithm
  ],

  // Optimization Goals
  goals: {
    responseTime: {
      target: 2000,
      unit: 'ms',
      description: 'AI response time under 2 seconds'
    },

    userSatisfaction: {
      target: 0.8,
```

```
      unit: 'percentage',
      description: '80% positive feedback'
    },

    conversionRate: {
      target: 0.15,
      unit: 'percentage',
      description: '15% of chats lead to orders'
    },

    sessionEngagement: {
      target: 5,
      unit: 'messages',
      description: 'Average 5 messages per session'
    },

    churnRate: {
      target: 0.05,
      unit: 'percentage',
      description: 'Less than 5% monthly churn'
    }
  }
});
```

Usage in Code

```
import { useProductLogger } from '@/lib/jnx-products';

// In API route
const logger = useProductLogger('qryx');

// Log installation
await logger.logEvent('shop_installed', {
  shop_id: shop.id,
  shop_domain: shop.shop_domain,
  plan: shop.plan_name
});

// Log chat message
await logger.logEvent('chat_message', {
  shop_id: shop.id,
  session_id: session.id,
  message_length: message.length,
  response_time: responseTime,
  model: 'gemini-2.0-flash-exp'
});

// Log conversion
if (order) {
  await logger.logEvent('conversion_tracked', {
    shop_id: shop.id,
    session_id: session.id,
    order_id: order.id,
    order_value: order.total,
    products: order.line_items.map(i => i.product_id)
  });
}
```

💰 Monetization (Shopify Billing API)

Pricing Tiers

```
const PRICING_TIERS = {
  trial: {
    name: 'Trial',
    price: 0,
    duration_days: 14,
    features: {
      monthly_messages: 100,
      analytics: 'basic',
      product_recommendations: true,
      customization: 'basic'
    }
  },

  basic: {
    name: 'Basic',
    price: 29,
    features: {
      monthly_messages: 1000,
      analytics: 'standard',
      product_recommendations: true,
      customization: 'standard',
      email_support: true
    }
  },

  pro: {
    name: 'Pro',
    price: 79,
    features: {
      monthly_messages: 5000,
      analytics: 'advanced',
      product_recommendations: true,
      customization: 'advanced',
      priority_support: true,
      custom_prompts: true
    }
  },

  enterprise: {
    name: 'Enterprise',
    price: 199,
    features: {
      monthly_messages: 'unlimited',
      analytics: 'enterprise',
      product_recommendations: true,
      customization: 'full',
      dedicated_support: true,
      custom_prompts: true,
      white_label: true
    }
  }
};
```

Billing Implementation

Create Subscription

```
// app/api/qryx/billing/subscribe/route.ts
export async function POST(request: Request) {
  const { shop, plan_tier } = await request.json();

  const shopRecord = await getShopByDomain(shop);
  const plan = PRICING_TIERS[plan_tier];

  // Create Shopify recurring charge
  const charge = await createRecurringCharge(shop, shopRecord.access_token, {
    name: `Qryx ${plan.name} Plan`,
    price: plan.price,
    return_url: `${process.env.APP_URL}/api/qryx/billing/confirm?shop=${shop}`,
    trial_days: plan_tier === 'trial' ? 0 : undefined
  });

  // Return confirmation URL
  return Response.json({
    confirmation_url: charge.confirmation_url
  });
}
```

Usage Tracking

```
// Track message usage
async function trackMessageUsage(shopId: string) {
  const shop = await getShop(shopId);
  const usage = await getCurrentMonthUsage(shopId);
  const limit = PRICING_TIERS[shop.plan_tier].features.monthly_messages;

  if (usage >= limit) {
    // Send upgrade notification
    await notifyShopOwner(shop, 'usage_limit_reached');
    return false; // Block message
  }

  return true; // Allow message
}
```

🚀 Deployment Strategy

Phase 1: MVP (Week 1-2)

```
✅ Core Features:
- Shopify OAuth
- Basic chat (Gemini 2.0 Flash)
- Admin dashboard (analytics)
- Storefront widget
- Free trial

❌ Not Yet:
- Advanced analytics
- Custom prompts
- A/B testing
```

Phase 2: Beta (Week 3-4)

```
✅ Add:
- Billing integration
- Product recommendations
- Order tracking
- Email notifications
- Basic customization

🎯 Goal: 10 beta shops
```

Phase 3: Launch (Week 5-6)

```
✅ Add:
- Advanced analytics
- Custom prompts
- Multi-language support
- Performance optimizations

🎯 Goal: Shopify App Store listing
```

Phase 4: Scale (Week 7+)

```
✅ Add:
- AI learning from feedback
- Auto-optimization
- White label option
- API for developers

🎯 Goal: 100+ paying shops
```

📋 Implementation Checklist

Backend Setup

- [ ] Database migration (schema above)
- [ ] Shopify OAuth endpoints
- [ ] Admin API routes
- [ ] Chat API endpoint
- [ ] Billing API integration
- [ ] Webhook handlers (app/uninstalled)
- [ ] JNX Learning Platform integration

Frontend

- [ ] Admin dashboard (embedded app)
- [ ] Storefront widget (vanilla JS)
- [ ] Widget styling
- [ ] Configuration UI
- [ ] Analytics charts

Shopify Integration

- [ ] App registration in Shopify Partners
- [ ] OAuth scopes configuration
- [ ] Script tag installation
- [ ] Webhook subscriptions
- [ ] Billing API setup

AI/ML

- [ ] Gemini 2.0 Flash integration
- [ ] System prompt engineering
- [ ] Product context building
- [ ] Intent recognition
- [ ] Product recommendation logic

Testing

- [ ] OAuth flow test
- [ ] Chat functionality test
- [ ] Widget rendering test
- [ ] Billing flow test
- [ ] Multi-shop test

Documentation

- [ ] Setup guide for shop owners
- [ ] API documentation
- [ ] Troubleshooting guide
- [ ] Privacy policy update
- [ ] Terms of service update

🔒 Security Considerations

Data Protection

1. **Encrypt access tokens** in database
2. **Validate HMAC** on all Shopify requests
3. **Rate limit** chat API (prevent abuse)
4. **Sanitize user input** (XSS prevention)
5. **GDPR compliance** (data export/deletion)

Best Practices

```
// Token encryption
import crypto from 'crypto';

function encryptToken(token: string): string {
  const cipher = crypto.createCipheriv(
    'aes-256-gcm',
    Buffer.from(process.env.ENCRYPTION_KEY!, 'hex'),
    crypto.randomBytes(16)
  );
  // ... encryption logic
}

// HMAC validation
function validateHMAC(params: URLSearchParams, hmac: string): boolean {
  const message = Array.from(params.entries())
    .filter(([key]) => key !== 'hmac')
    .sort(([a], [b]) => a.localeCompare(b))
    .map(([key, val]) => `${key}=${val}`)
    .join('&');

  const generatedHMAC = crypto
    .createHmac('sha256', process.env.SHOPIFY_API_SECRET!)
    .update(message)
    .digest('hex');

  return crypto.timingSafeEqual(
    Buffer.from(hmac),
    Buffer.from(generatedHMAC)
  );
}
```

📊 Success Metrics

Key Performance Indicators

Product Metrics:
- Shops installed
- Active shops (used in last 30 days)
- Churn rate
- Average revenue per shop (ARPU)

User Engagement:
- Chat sessions per shop per day
- Messages per session
- Customer satisfaction rating
- Conversion rate (chats → orders)

Technical Metrics:
- Response time (p50, p95, p99)
- Uptime (99.9% target)
- Error rate (<1%)
- API latency

Learning Platform:
- Events logged per day
- AI insights generated
- Optimizations deployed
- Performance improvements

🎉 Summary

Qryx ist eine **production-ready Shopify App**, die:

✅ **Multi-Tenant** (jeder Shop = Tenant)
✅ **JNX-OS powered** (Auth, DB, Learning)
✅ **AI-driven** (Gemini 2.0 Flash)
✅ **One-Click Install** (Shopify OAuth)
✅ **Monetisiert** (Shopify Billing API)
✅ **Skalierbar** (Event-basiertes Learning)
✅ **GDPR-compliant**

Next Steps:
1. Database Migration ausführen
2. Shopify Partner Account erstellen
3. OAuth Flow implementieren
4. MVP bauen (2 Wochen)
5. Beta Testing (10 Shops)
6. App Store Launch

Version: 1.0.0
Last Updated: 2024-12-28
Status: Ready to Build 🚀